

Outline

- General organization of the course
- Object oriented programming

Basic types and their operations

- Methods
- UML class diagrams

- A data type (short: type) defines a set of possible values and the possible operations (methods) on these types
- All variables and attributes have a type (and methods have a return type and types for the arguments)
- In Java, classes define data types
- Java also provides eight basic types (e.g. **int** and **double**)
- Each value of a basic type requires the same amount of space to represent it in the computer
- The space is measured in bits
(How many values can be represented with n bits?)

Integer based types



Type	Space	Smallest value	Highest value
byte	8	-128	127
short	16	-32 768	32 767
int	32	-2 147 483 648	2 147 483 647
long	64	-9 223 372 036 854 775 808	9 223 372 036 854 775 807

Using smaller types like byte or short saves space

Note: Java does **not** warn in case of overflow or underflow!

Overflow example

```
int x = 2147483647; // largest int value
x = x + 1;           // overflow
System.out.println(x);
```

prints: **-2147483648**  Smallest **int** value

- You can assign a first value to a variable when you declare it directly (initialization)
- You can even declare it at the point where you need it for the first time

Floating point numbers

- There are two kinds of floating point numbers

Type	Space	Smallest value	Highest value	Precision
float	32	$-3.4e+38$	$3.4e+38$	6 to 7 decimal digits
double	64	$-1.7e+308$	$1.7e+308$	15 to 16 decimal digits

- Both double and float are approximate types, they are **not** precise
- Overflow/Underflow returns the values **Infinity** or **-Infinity**
- Consider the accuracy of the result to select the right type
- Floating point constants in the program are understood as `double`
- To differentiate, append the letter **f** (or **F**) or **d** (or **D**)

Example: `3.14f`

Other basic types

Type	Space	Values
boolean	1	true, false
char	16	all unicode characters

- **Unicode** is a character set that includes all alphabets commonly used anywhere in the world, for **example**
 - The characters of our keyboard (including umlauts)
 - Chinese characters
 - Egyptian hieroglyphics
 - ...
- Char constants are written with single quotation marks: 'A', ';', '\n'

Basic types and constructors



- Because they are used often and their implementation is efficient, no constructor has to be called
- It is sufficient, for **example**, to write **int x = 2;**
- Only one memory area is reserved (for the value), not two as in the case of reference types (reference + object)

Equality and identity

- When are two objects of a basic type "equal"?
 - For basic types the value is stored directly, without reference
- Check the equality of the value for basic types therefore with `==`
- For reference types the reference is compared with `==`

Example

```
int i1 = 5;
int i2 = 6;
int i3 = 5;
System.out.println(i1 == i2);    // false
System.out.println(i1 == i3);    // true
```

```
Point p1 = new Point();
Point p2 = new Point();
Point p3 = p1;
System.out.println(p1 == p2);    // false
System.out.println(p1 == p3);    // true
```


Basic operations

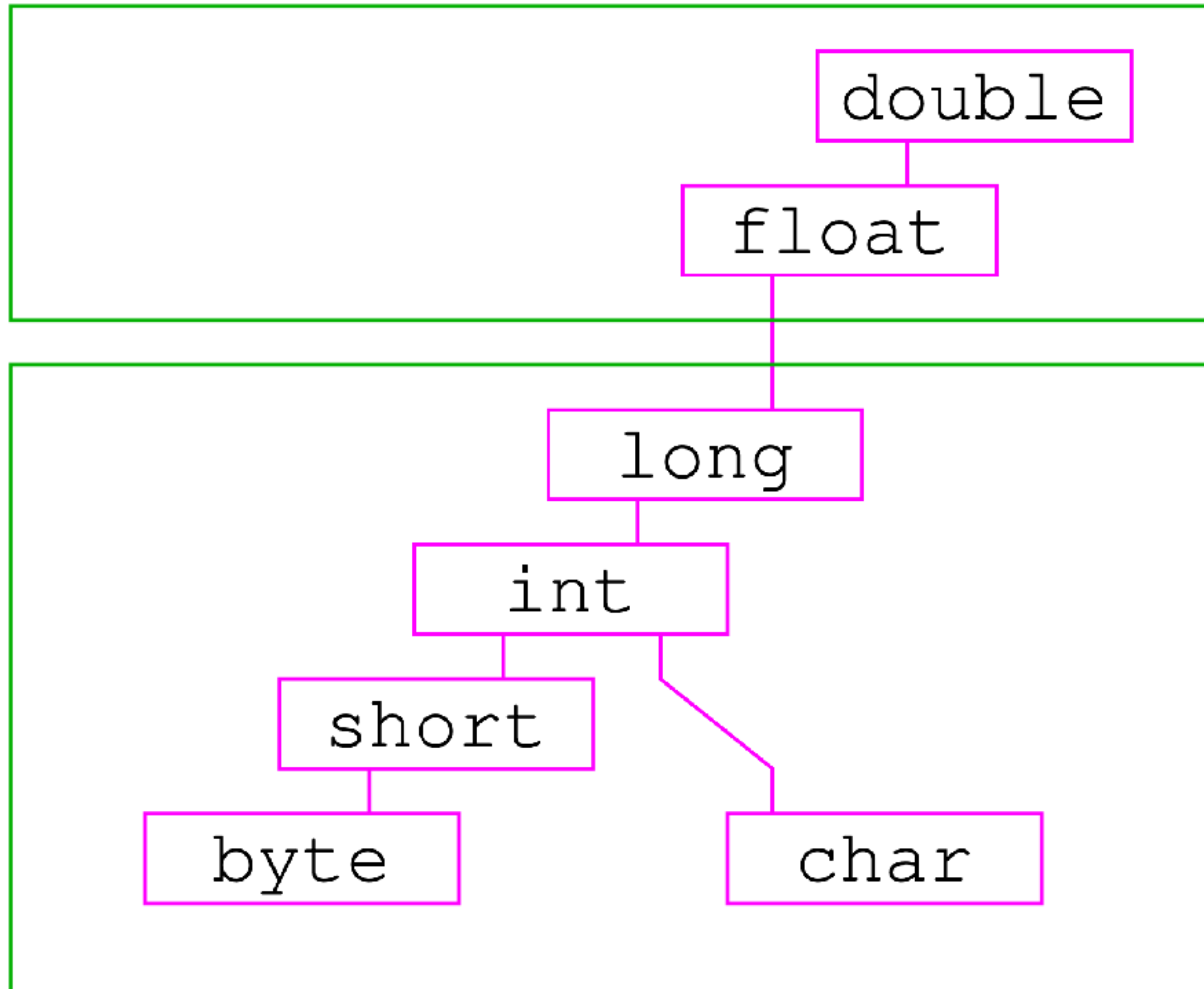


- The operators **+**, **-**, *****, **/** and **%** exist for each of the number types
- If they are applied to a pair of arguments of different types, the more specific one is converted to the more general one beforehand (**implicit type cast**)

Overview of basic types

Floating point numbers

Integers



Example

```
short xs = 1;  
int x = 9999999;  
System.out.println(x + xs);
```

Prints the **int** value 10000000

```
float xs = 1.0f;  
int x = 999999999;  
System.out.println(x + xs);
```

Can output floating
point numbers

Prints the **float** value 1.0E9

Elementary operations

Precedence	Operator	Description
Arithmetic and comparison operators		
1	+x, -x	unary plus/minus
2	x * y, x / y, x % y	multiplication, division, modulo
3	x + y, x - y	addition, subtraction
4	x < y, x <= y, x > y, x >= y	size comparisons
6	x == y, x != y	equality, inequality
Operators on integers		
1	~x	Bitwise complement (NOT)
Operators on integers and truth values		
7	x & y	Bitwise AND
8	x ^ y	Bitwise either/or (XOR)
9	x y	Bitwise or (OR)
Operators on truth values		
1	!x	NOT
10	x && y	Sequential AND
11	x y	Sequential OR
Operators on integers		
4	x << y	Left shift
4	x >> y	Right shift (sign conform)
4	x >>> y	Right shift (without sign)

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html>

Example



```
String s0 = "";  
String s1 = "Hel";  
String s2 = "lo Wo";  
String s3 = "rld!";  
System.out.println(s0 + s1 + s2 + s3);
```

... writes **Hello World!** on the output

Implicit conversion to **String**

- Every value in Java has a representation as a **String**
- If the operator "+" is applied to a value of type **String** and another value x, then x is automatically converted to its **String** representation
- This is a simple way to output float or double
- Example

```
double x = -0.55e13;  
System.out.println("A floating point number: " + x);
```

Output on the console: A floating point number: -5.5E12

Exercise: what will the following program output?

```
public class Test {  
    public static void main(String[] args) {  
        int x = 2, y = 5;  
  
        int exp1 = (x * y / x);  
        int exp2 = (x * (y / x));  
  
        System.out.println(exp1);  
        System.out.println(exp2);  
    }  
}
```

Solution

Exercise: what will the following program output?

```
public class Test {  
    public static void main(String[] args) {  
        int x = 10, y = 5;  
  
        int exp1 = (y * (x / y + x / y));  
        int exp2 = (y * x / y + y * x / y);  
  
        System.out.println(exp1);  
        System.out.println(exp2);  
    }  
}
```

Solution

Exercise: what will the following program output?

```
public class Test {  
    public static void main(String[] args) {  
        int x, y, z;  
        x = y = z = 2;  
        x += y;  
        y -= z;  
        z /= (x + y);  
        System.out.println(x + " " + y + " " + z);  
    }  
}
```

Solution